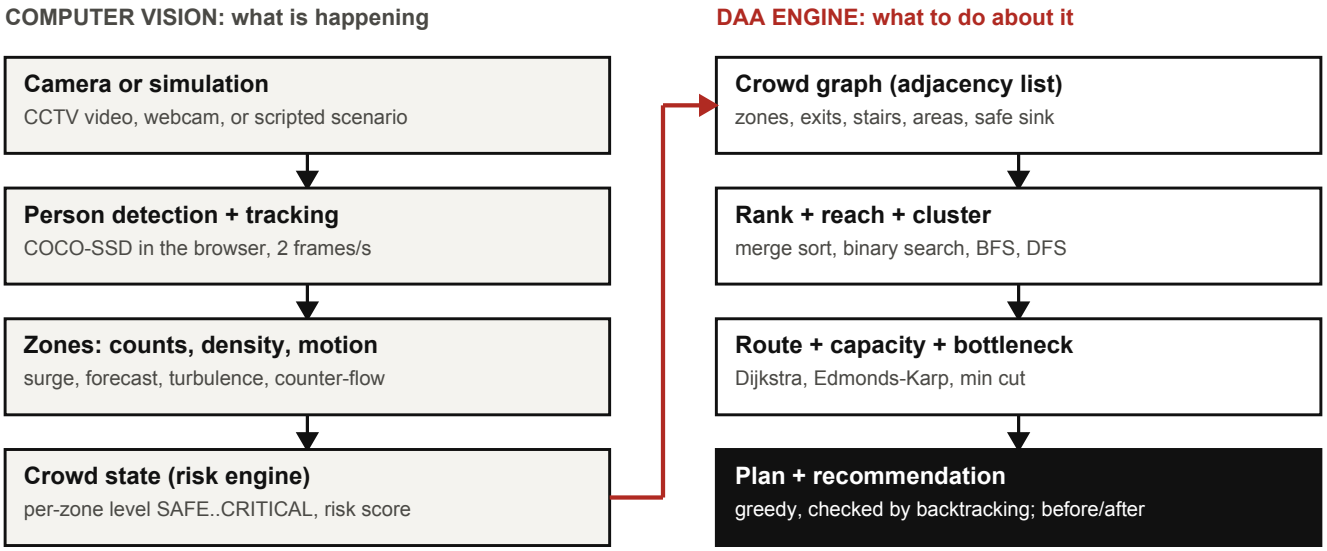
SENTINEL

Algorithmic Crowd Safety and Evacuation Optimization System

Project report: how the system works, the DAA concepts inside it and why we chose them, and what comes next.

Course: CS3212 Design and Analysis of Algorithms
Domain: Computer Vision: intelligent crowd analysis and management
Application: Crowd management and emergency evacuation

Computer vision detects what is happening in the crowd. Design and analysis of algorithms decides what should be done about it.



Contents

1	The problem and what Sentinel does	6	Complexity analysis, comparison and benchmark
2	How the whole system works	7	Testing and how to verify the results
3	From a crowd to a graph	8	What we will add next
4	DAA concepts: what, why and how (10 algorithms)	A–C	Demo script, viva questions, how to run
5	End-to-end worked example: Emergency Crowd Surge		

PART 1

The problem and what Sentinel does

Crowd crushes happen when people pack into a space faster than they can leave it. Once density passes roughly five people per square metre, people can no longer choose where they move; pressure waves pass through the crowd and can injure or kill. Railway platforms, stairs, temple queues, stadium gates and concerts are typical places. The warning signs are visible on CCTV minutes before the danger peaks: one area filling much faster than the rest, people pushed in different directions (turbulence) and two streams moving against each other (counter-flow).

Control rooms have cameras but not decision support. An operator watching twenty screens must answer, under time pressure: **Which area is most dangerous? Can those people still reach an exit? Which way should we send them? How fast can the area be emptied, and what limits that? Which action helps most?**

Sentinel answers these questions in two layers:

- **Computer vision layer** (the original Sentinel Crowd project): detects people in video, counts them per zone, measures movement, and raises explainable alerts (density, surge, forecast, turbulence, counter-flow).
- **DAA decision layer** (this course project): turns the measured crowd state into a weighted graph and runs classic algorithms with known correctness and complexity to decide what to do, and shows the measured improvement.

Objectives

1. Measure the crowd state per zone from video, or from a deterministic simulation for repeatable demos.
2. Represent the venue as a weighted graph generated from that live state, not a separate toy graph.
3. Find reachable exits (BFS), the connected danger region (DFS) and the safest route (Dijkstra).
4. Compute evacuation capacity (maximum flow) and the bottleneck that limits it (minimum cut).
5. Choose interventions quickly (greedy) and check them against the exact optimum (backtracking).
6. Show a before/after improvement that is computed, and make every result traceable: input → algorithm → output → decision.

PART 2

How the whole system works

Sentinel is a single-page web app (React + Vite) that runs entirely in the browser: no server, no API key, and video never leaves the device. The code is split into layers so the algorithms can be tested without a camera or a browser.

Layer	What it does	Main files
Vision	Loads COCO-SSD (TensorFlow.js) and detects people twice per second in a webcam or uploaded MP4. A light tracker follows each person for a few seconds to estimate velocity.	`lib/detector.js`, `lib/tracker.js`, `hooks/useLiveDetection.jsx`
Zones	Splits the view into named zones (default 4×3 grid A1..C4) and aggregates people and motion per zone, including a counter-flow index.	`lib/zones.js`
Risk engine	CrowdEngine: smoothing, density levels, surge, forecast, turbulence and counter-flow rules; produces a snapshot with levels and a 0–100 risk score.	`lib/risk.js`
Simulation	A scripted, deterministic 90 s station scenario (Platform 2 of a junction) plus three side cameras; what-if actions change it.	`lib/simulation.js`
Site	Where the exits and entries are (Gate 1–3, footbridge stairs). Advice names real exits only when a site is configured.	`lib/site.js`
DAA engine	Builds the crowd graph and runs the decision pipeline; projects risk with the existing model.	`engine/*.js`
Algorithms	Hand-written data structures and algorithms; no graph, routing or sorting libraries.	`algorithms/**`
UI	Monitor, Station, Zones, Timeline, Setup, DAA Engine and Algorithm Analysis tabs.	`components/**`

2.1 Computer vision: from pixels to crowd state

Each sampled frame goes through the person detector. Every detected person becomes a point (the centre of the box). The tracker matches points between frames to estimate speed and direction. Points are counted into zones. Per zone, Sentinel computes the average velocity, the spread of velocities, and a counter-flow index (0 when everyone moves the same way, 1 when two equal streams move in opposite directions along one line).

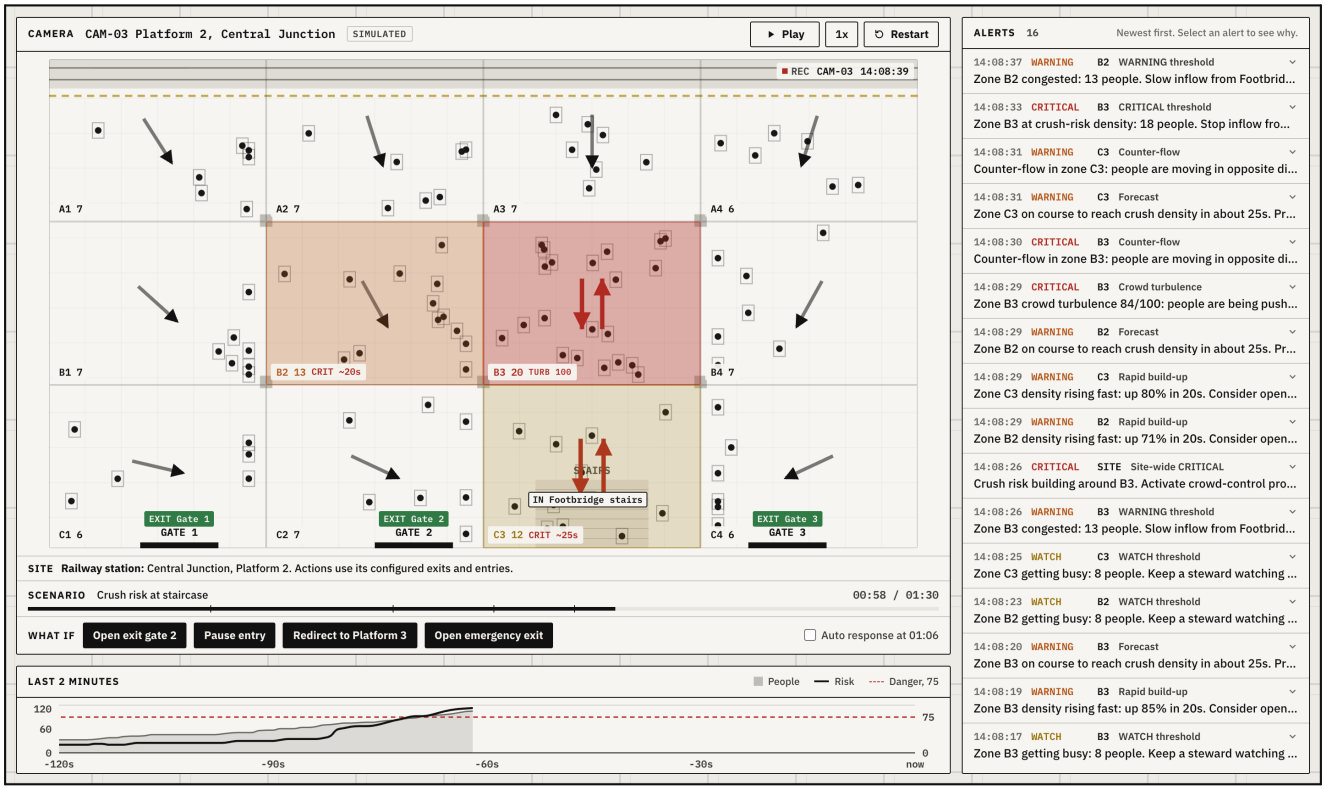
2.2 The risk rules

Rule	Meaning	Why it matters
Density	People per zone vs WATCH / WARNING / CRITICAL limits, scaled to zone size	The basic crush indicator
Surge	A zone rising by a set % and number of people within 20 s	Fast build-up gives little time to react
Forecast	Least-squares growth rate projected to the CRITICAL threshold	Early warning: seconds until crush density
Turbulence	Density × spread of movement (Helbing et al., 2007)	Pushing in different directions precedes crushes
Counter-flow	Two streams moving opposite ways along one line	Common trigger at stairs and gates

Overall risk is the worst zone's level, raised by one level while a surge, turbulence or counter-flow is active. Every alert explains which rule fired. The DAA engine reads this snapshot as its input.

2.3 Simulation and the Emergency Crowd Surge scenario

For a reliable classroom demo, Simulation mode plays a scripted 90-second evening rush: a train is delayed, the crowd funnels toward the staircase in zone B3, a train arrives and passengers push up the stairs against the crowd. **Load Emergency Crowd Surge** replays this script with no automatic response to 0:58 and pauses. No random numbers are used, so every load gives exactly the same state: **105 people**, zone B3 at **111% of crush density**, counter-flow at the foot of the stairs, Gate 2 and the emergency exit closed.



Monitor tab at 0:58 in the Emergency Crowd Surge scenario. B3 is CRITICAL with turbulence 100 and counter-flow (red opposing arrows); B2 and C3 are forecast to reach crush density in about 20–25 s. The alert feed explains each rule.

2.4 The tabs

Tab	Purpose
Monitor	Camera view (simulated or live), zone grid, flow arrows, alert feed with reasons, what-if buttons
Station	Four-camera station map coloured by risk; arrows show where actions send people
Zones / Timeline	Per-zone details; full chart, alert history, incident report, CSV export
Setup	Thresholds, notifications, site exits/entries, zone editor
DAA Engine	The decision pipeline, eight steps, each drawn on the crowd graph with step-by-step replay
Algorithm Analysis	Complexity table (theory vs observed runtime) and benchmark mode

PART 3

From a crowd to a graph

Every DAA algorithm in Sentinel works on **one graph**, rebuilt from the live crowd state (engine/crowdGraphBuilder.js). In the emergency scenario it has **V = 21 nodes and E = 48 directed edges**.

Node type	Where it comes from	Data on the node
Zone (12)	The camera's zones A1..C4	people, density (% of crush density), risk = max(alert level, turbulence, counter-flow)
Exit (Gate 1–3, emergency exit)	Site configuration; linked to the nearest zone	closed or open (a closed exit has capacity 0)
Stairs	Site entry 'Footbridge stairs'	counter-flow measured by the camera at its foot
Area (footbridge, concourse, Platform 3)	The other three station cameras	people and density from those cameras
Safe area	One sink node: the street	-

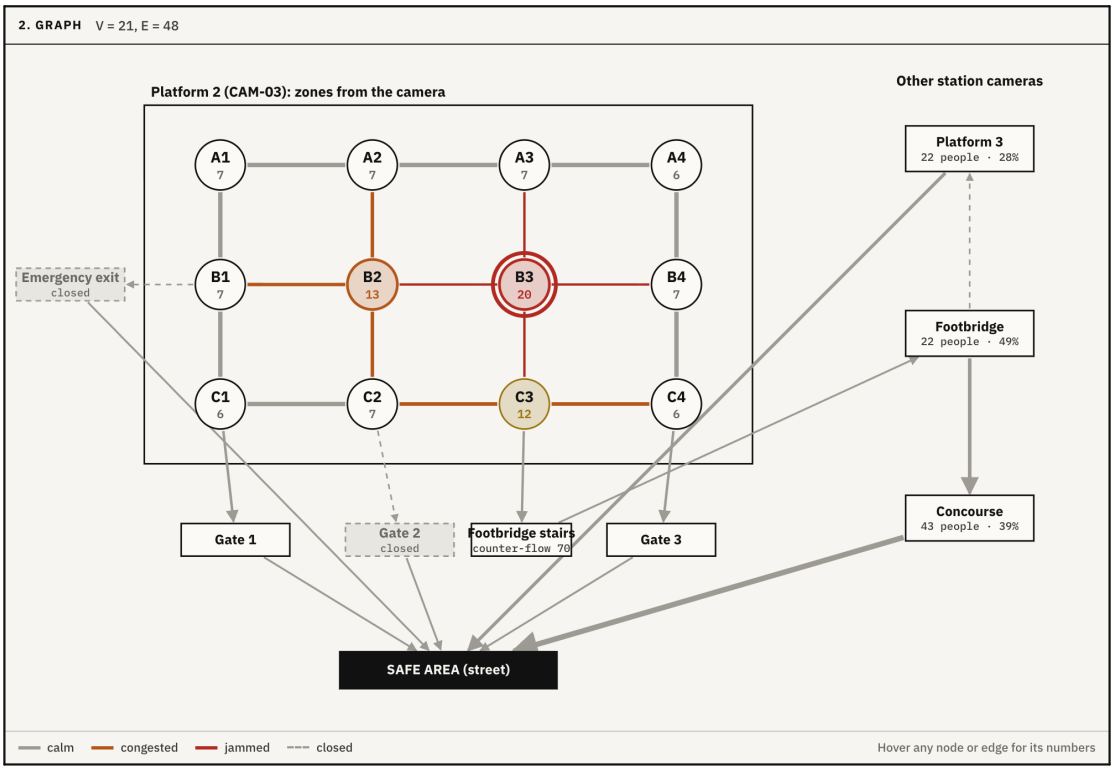
Edge attribute	Meaning	How it is computed
distance	Metres walked	Zone centres on an 80 m × 24 m platform; fixed lengths for stairs and passages
capacity	People per minute that can pass safely	90/min on open floor × crowd factor; gates 40–45; stairs 60 × (1 − 0.6 × counter-flow); 0 if closed
congestion	How full the destination is	Destination density in % of crush density (from the camera)
risk	Danger at the destination	max(alert level, turbulence, counter-flow), 0–100
cost	Crowd-aware cost for Dijkstra	0.4 × distance + 0.4 × congestion + 0.2 × risk

```
cost = 0.4 × distance(m) + 0.4 × congestion(%) + 0.2 × risk(0–100)
```

Crowd factor. Pedestrian flow slows in dense crowds (the fundamental diagram of pedestrian flow). Sentinel uses a simple version: full capacity up to 50% of crush density, falling linearly to 40% of capacity at crush density. **Counter-flow loss.** At full counter-flow the stairs lose 60% of their capacity, because people going down block people going up.

Why a graph, and why an adjacency list?

A station is a network: areas connected by corridors, stairs and gates, each with a length and a width. A graph captures exactly this, and once the crowd is a graph, decades of proven algorithms apply. We store it as an **adjacency list** (each node keeps a list of its outgoing edges). Our graph is sparse: 48 edges against $V^2 = 441$ possible pairs. An adjacency list uses $O(V + E)$ memory and lets BFS, DFS, Dijkstra and max flow visit a node's neighbours in time proportional to its degree, which is what gives them their $O(V + E)$-style bounds. An adjacency matrix would need $O(V^2)$ memory and $O(V)$ time to list neighbours.



The crowd graph in the app (DAA Engine, step 2). Circles are camera zones with their head counts (colour = alert level). Edge colour shows congestion; dashed grey edges are closed (capacity 0). Right column: areas measured by the other station cameras.

Time	`O(V + E)`
Space	`O(V + E)`
Data structure	Adjacency list
Observed (this scenario)	0.049 ms per run, averaged over repeated runs
Source file	`src/engine/crowdGraphBuilder.js`

PART 4

DAA concepts: what, why and how

Each algorithm below is hand-written in src/algorithms/ and actually called by the decision engine (engine/decisionEngine.js). For each one we give: what it is, **why we use it in Sentinel**, what it computes in the emergency scenario, its complexity, and why we did not choose an alternative.

#	Question the operator asks	Algorithm	Answer in the emergency scenario
1	Which zones are most dangerous?	Merge sort + binary search	B3 (111%), band CRITICAL
2	Which exits can those people reach?	BFS	Gate 1, Gate 3, Footbridge stairs; not Gate 2, Emergency exit
3	Which connected zones are affected?	DFS	B3 → B2 → C3
4	Which way should they go?	Dijkstra	B3 → B4 → C4 → Gate 3 → Safe area (street)
5	How fast can we evacuate?	Edmonds-Karp max flow	115 people/min
6	What limits evacuation?	Min cut	C1 → Gate 1, C3 → Footbridge stairs, C4 → Gate 3
7	What should we do?	Greedy (+ backtracking check)	Open Gate 2 + Open platform-end emergency exit

4.1 Merge sort: ranking zones by danger

What it is. A divide-and-conquer sort. Split the list in half, sort each half recursively, then merge the two sorted halves by repeatedly taking the smaller front item. The recurrence T(n) = 2T(n/2) + O(n) solves to O(n log n).

Why we use it in Sentinel. Before deciding anything we need to know where the danger is. Ranking zones by density (ties broken by people, then turbulence) gives the operator a priority list and gives the rest of the pipeline its starting point: the top zone becomes the source for BFS, DFS and Dijkstra. Merge sort is **stable**, so zones with equal density keep their camera order and the ranking is reproducible, and it is O(n log n) in the **worst** case, which matters when a stadium has hundreds of zones.

What it computes here. Input: 12 zones in camera order. Output: B3 111%, B2 72%, C3 67%, A1 39%, A2 39%, … (11 divides and 11 merges, i.e. n − 1 merges for n = 12).

Time	`O(n log n)`
Space	`O(n)`
Data structure	Arrays (merge buffers)
Observed (this scenario)	1.6 µs per run, averaged over repeated runs
Source file	`src/algorithms/sorting/mergeSort.js`

Why not something else? Quicksort is O(n²) in the worst case and not stable. Insertion sort is O(n²). JavaScript's Array.prototype.sort hides the algorithm and its behaviour depends on the engine, so it is deliberately not used for the ranking.

> *Viva answer: "Merge sort ranks the zones in guaranteed O(n log n), and because it is stable, equal zones stay in a predictable order."*

4.2 Binary search: finding the alert band

What it is. Search in a sorted array by comparing with the middle element and discarding half of the range each step: O(log n).

Why we use it in Sentinel. Alert thresholds are a sorted list (SAFE from 0, WATCH from 8, WARNING from 13, CRITICAL from 18 people per zone area). For each zone we need the last threshold that the density reaches: a classic 'last element ≤ value' search. With four bands the saving is small, but the same code scales to fine-grained tables (per-zone capacity tables, many alert bands) where a linear scan would be wasteful. It uses exactly the same rule as the existing risk engine, so the two always agree.

What it computes here. Zone B3: 20 people per zone area against thresholds [0, 8, 13, 18]. Probes: index 1 (8 ≤ 20) → index 2 (13 ≤ 20) → index 3 (18 ≤ 20). Found **CRITICAL** in 3 comparisons.

Time	`O(log n)`
Space	`O(1)`
Data structure	Sorted array of thresholds
Observed (this scenario)	0.7 µs per run, averaged over repeated runs
Source file	`src/algorithms/searching/binarySearch.js`

Why not something else? Linear search is O(n). A hash map cannot answer 'which range contains this value'. Binary search is the standard tool for range lookup in sorted data.

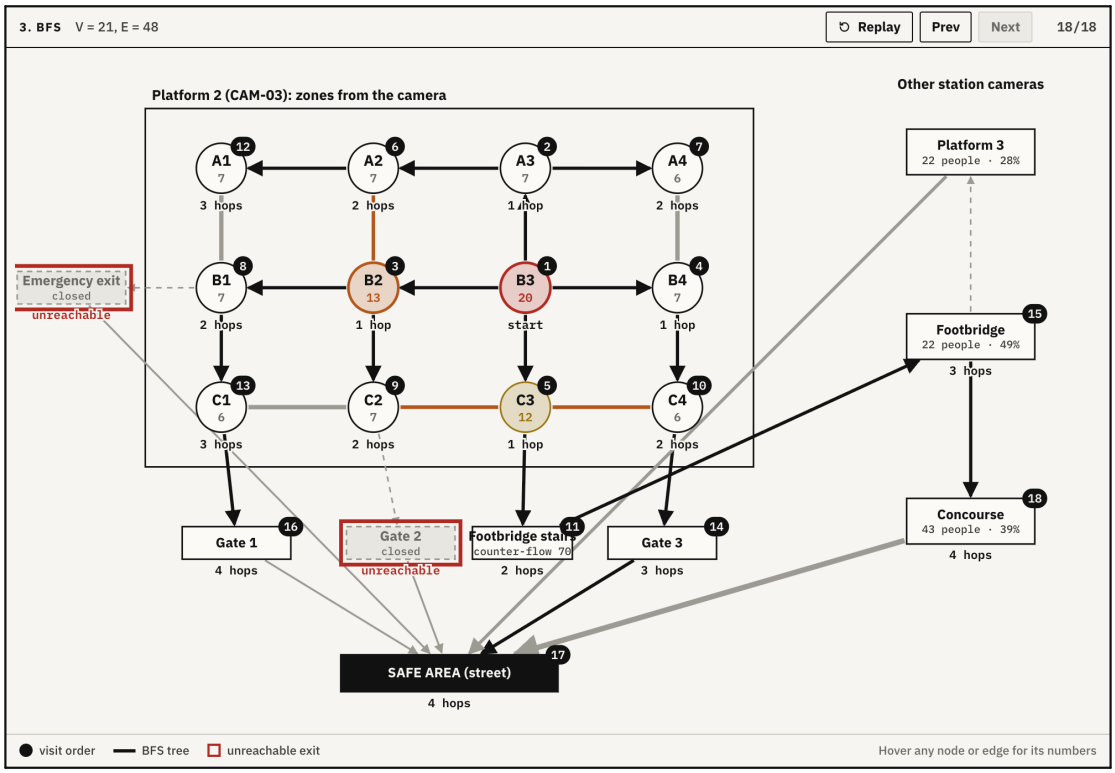
> *Viva answer: “The thresholds are sorted, so binary search finds a zone's alert band in O(log n) comparisons instead of checking every threshold.”*

4.3 Breadth-first search: which exits are reachable?

What it is. Explore a graph level by level from a start node using a FIFO **queue**: first all neighbours, then their neighbours, and so on. Each node is visited once and each edge checked once.

Why we use it in Sentinel. Before routing people we must know whether an exit can be reached at all. BFS answers reachability, and because it explores in hop order it also gives the fewest-hops distance to each exit. Closed edges (capacity 0, e.g. a locked gate) are skipped, so a locked exit shows up as **unreachable**, which immediately suggests an intervention: open it.

What it computes here. From B3, BFS visits 18 nodes in the order B3 → A3 → B2 → B4 → C3 → A2 → A4 → B1 → C2 → C4 → … Reachable exits: Gate 1 (4 hops), Gate 3 (3 hops), Footbridge stairs (2 hops). **Not reachable: Gate 2, Emergency exit** (closed).



BFS in the app: numbers show the visit order, black edges the BFS tree, labels the hop count. Gate 2 and the emergency exit are marked unreachable.

Time	`O(V + E)`
Space	`O(V)`
Data structure	Queue
Observed (this scenario)	0.011 ms per run, averaged over repeated runs
Source file	`src/algorithms/graph/bfs.js`

Why not something else? DFS would also find reachability but not the hop distance. Dijkstra is unnecessary when we only ask 'can we get there'.

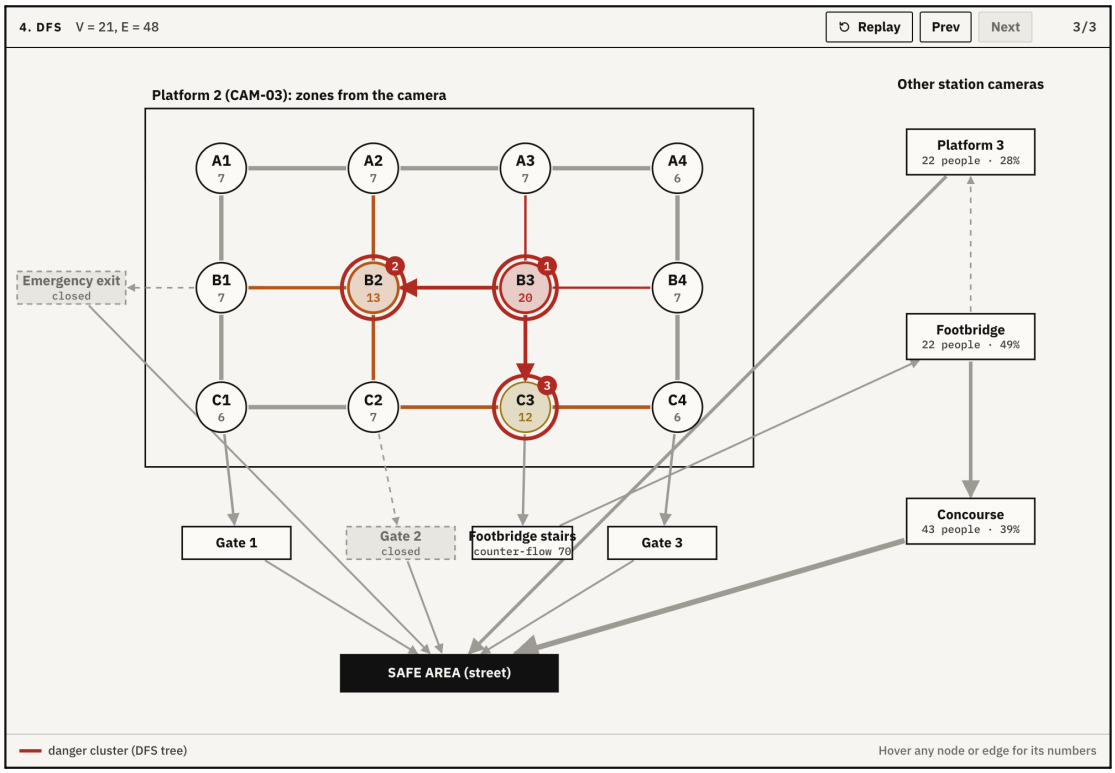
> *Viva answer: "BFS checks reachability through the crowd network in O(V + E). Closed gates have zero capacity, so BFS correctly reports them as unreachable."*

4.4 Depth-first search: the affected danger cluster

What it is. Explore as deep as possible along one branch before backtracking. We use an explicit **stack** instead of recursion, so a large venue cannot overflow the call stack.

Why we use it in Sentinel. Danger rarely stays in one zone. Stewards need the whole **connected region** of crowded zones around the critical zone, because it must be managed together: moving people from B3 into an already crowded neighbour just moves the problem. DFS with a filter (enter only zones at WATCH or above) is a flood fill that returns exactly that region.

What it computes here. From B3, entering only zones at WATCH or above: **B3** → **B2** → **C3**. These zones hold the crowd that must be handled as one area; the calm zones around them are where people can be sent.



DFS danger cluster: the red tree from B3 enters only zones at WATCH or above.

Time	`O(V + E)`
Space	`O(V)`
Data structure	Stack (explicit)
Observed (this scenario)	1.5 µs per run, averaged over repeated runs
Source file	`src/algorithms/graph/dfs.js`

Why not something else? BFS would find the same set; DFS is the natural fit for region filling and only needs to remember the current path. Union-Find could maintain clusters incrementally as zones change level (see Part 8).

> *Viva answer: "DFS explores the connected region of dangerous zones around the critical zone, in O(V + E), using an explicit stack."*

4.5 Dijkstra: the safest evacuation route

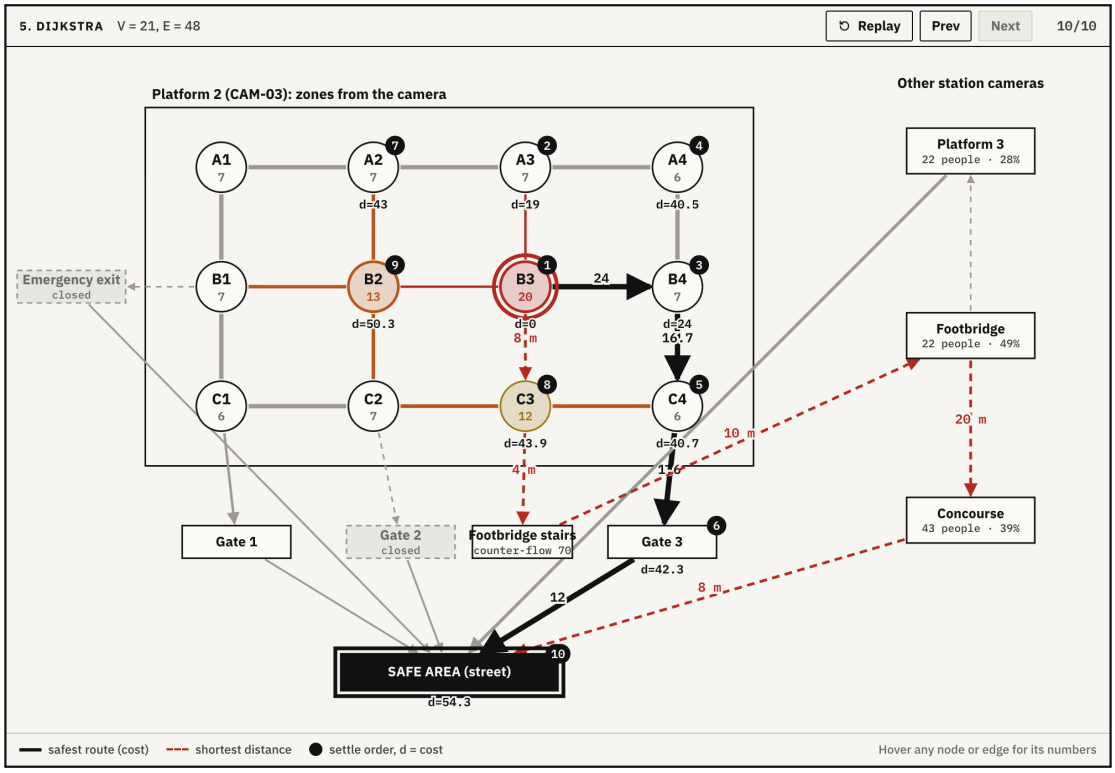
What it is. Single-source shortest paths for graphs with non-negative edge weights. It repeatedly settles the unsettled node with the smallest known distance, taken from a **binary min-heap** (priority queue), and relaxes its outgoing edges. Correct because, with non-negative weights, no later path can make a settled node cheaper.

Why we use it in Sentinel. The shortest route is not always the safest. Sentinel's edge cost combines distance, congestion and risk, all measured by the cameras, so Dijkstra finds the route that avoids crowded and turbulent areas, even if it is longer. All costs are ≥ 0, which is exactly the condition Dijkstra needs. To show the difference, the engine runs the same Dijkstra twice: once on metres and once on cost.

What it computes here. Two routes from B3 on the same graph:

Route	Path	Metres	Cost	Max congestion	Max risk
Shortest (by metres)	B3 → C3 → Footbridge stairs → Footbridge → Concourse → Safe area (street)	50	129.5	67%	70
Safest (by cost)	B3 → B4 → C4 → Gate 3 → Safe area (street)	62	54.3	39%	2

The safest route is 12 m longer but its cost is 75.2 lower, because the short route climbs the stairs through C3 (67% congestion) against counter-flow (risk 70). Settle order: B3 0, A3 19, B4 24, A4 40.5, C4 40.7, Gate 3 42.3, A2 43, C3 43.9, …



Dijkstra in the app: solid black = safest route with edge costs; dashed red = shortest route by metres; numbers give the settle order and d = cost from B3.

Time	`O((V + E) log V)`
Space	`O(V + E)`
Data structure	Binary min-heap (priority queue)
Observed (this scenario)	0.011 ms per run, averaged over repeated runs
Source file	`src/algorithms/graph/dijkstra.js`
Lazy deletion	Instead of decrease-key, a node is pushed again when its distance improves; stale heap entries are skipped when popped. Same O((V + E) log V) bound because log E ≤ 2 log V.

Why not something else? BFS ignores weights. Bellman-Ford handles negative weights we do not have and costs O(V·E). Floyd-Warshall computes all pairs in $O(V^3)$, which we do not need. A* needs a heuristic and is planned for large venues (Part 8).

> *Viva answer: "Dijkstra is used because evacuation costs are never negative. The cost combines distance, congestion and risk, so it picks a longer route when the shorter one runs through the crush."*

4.6 Maximum flow (Edmonds-Karp): evacuation capacity

What it is. A flow network has a source, a sink and edge capacities. The maximum flow is the largest amount that can move from source to sink without exceeding any capacity and with flow conserved at every node. **Ford-Fulkerson** repeatedly finds a source-to-sink path with spare capacity in the **residual graph** and pushes as much as the path's tightest edge allows. **Edmonds-Karp** always picks the *shortest* such path (by BFS), which bounds the number of augmentations to O(V·E).

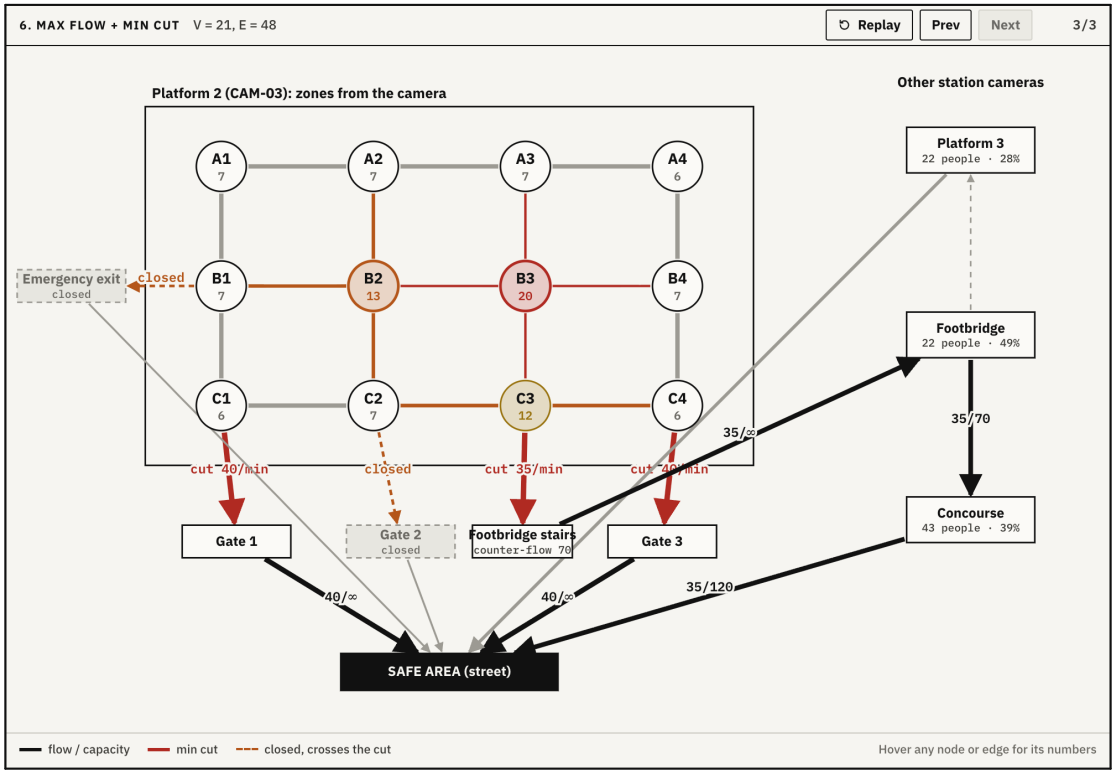
Why we use it in Sentinel. People moving through corridors, stairs and gates behave like flow through pipes with limited width: each connection lets only so many people per minute through. Edge capacity in Sentinel means exactly that. A virtual source feeds every occupied zone and the sink is the street, so the maximum flow is the **evacuation capacity of the whole station in people per minute**. Dividing the people on the platform by it gives an estimated clearance time.

What it computes here. Three augmenting paths, found by BFS on the residual graph:

Iteration	Augmenting path (from the crowd)	Pushed	Total
1	C1 → Gate 1 → Safe area (street)	+40	40
2	C4 → Gate 3 → Safe area (street)	+40	80
3	C3 → Footbridge stairs → Footbridge → Concourse → Safe area (street)	+35	115

Maximum flow = 115 people/min. With 105 people on the platform, estimated clearance = 105 ÷ 115 × 60 ≈ **55 s**.

The residual graph. Each edge u→v with capacity c becomes a forward arc with residual c − f and a reverse arc v→u with residual f. Reverse arcs let a later path cancel flow that an earlier path sent the wrong way, which is what makes the greedy path choice end at the true maximum. In the code, arc i and arc i XOR 1 are always a forward/reverse pair.



Max flow in the app: black edges show flow / capacity; thick red edges are the min cut (labelled with capacity); dashed orange edges are closed exits that cross the cut.

Time	`O(V · E²)`
Space	`O(V + E)`
Data structure	Residual graph + queue (BFS)
Observed (this scenario)	0.083 ms per run, averaged over repeated runs
Source file	`src/algorithms/flow/maxFlow.js`

Why not something else? Plain Ford-Fulkerson with arbitrary paths runs in O(E·f), which depends on the capacity values (and may not terminate with irrational capacities). Edmonds-Karp's bound depends only on the graph size. Dinic's algorithm, $O(V^2 \cdot E)$, is faster for very large graphs (Part 8).

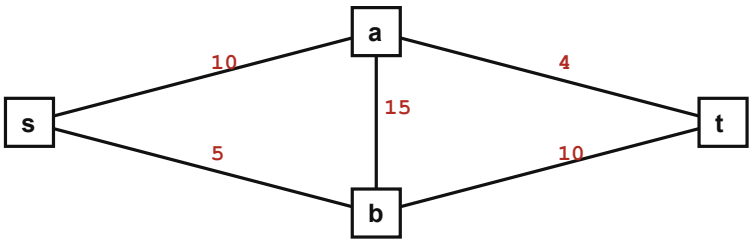
> *Viva answer: "Crowd movement through corridors is flow through capacity-limited edges. Edmonds-Karp augments along shortest paths with BFS, so it runs in $O(V \cdot E^2)$."*

4.7 Minimum cut: the bottleneck

What it is. An s-t cut splits the nodes into a source side and a sink side; its capacity is the total capacity of edges going from the source side to the sink side. The **max-flow min-cut theorem** says the maximum flow equals the capacity of the smallest cut. After Edmonds-Karp, the nodes still reachable from the source in the residual graph form the source side of a minimum cut.

Why we use it in Sentinel. Max flow says how much; min cut says **why not more**. The cut edges are the bottleneck of the evacuation, which tells the operator where to act: only adding capacity across the cut can increase evacuation capacity. Sentinel also lists **closed** edges that cross the cut (a locked gate next to the crowd), because opening one adds capacity directly across the bottleneck.

What it computes here. Cut edges: **C1 → Gate 1** (40/min), **C3 → Footbridge stairs** (35/min), **C4 → Gate 3** (40/min). Total 115/min = the max flow, as the theorem requires. The stairs carry only 35/min because the camera measures 70% counter-flow at their foot. Closed edges crossing the cut: B1 → Emergency exit, C2 → Gate 2.



Max flow s→t = 14 (a→t 4 + b→t 10)

Min cut = {a→t, b→t}: capacity 4 + 10 = 14

Raising s→a does nothing: it is not in the cut.

Toy example of the theorem and the key insight used by the greedy step: improving an edge that is not in the min cut cannot raise the maximum flow.

Time	`O(V + E) after max flow`
Space	`O(V)`
Data structure	Residual graph + queue (BFS)
Observed (this scenario)	0.013 ms per run, averaged over repeated runs
Source file	`src/algorithms/flow/minCut.js`

Why not something else? Checking every possible cut is exponential (2^V subsets). Reading the cut from the final residual graph costs one BFS.

Viva answer: “By max-flow min-cut, the saturated edges leaving the residual-reachable set limit the total flow, so they are the evacuation bottleneck.”

4.8 Greedy: choosing interventions

What it is. A greedy algorithm builds a solution step by step, always taking the option that looks best right now, and never undoing a choice. It is fast and simple, but only guaranteed optimal for problems with the greedy-choice property (e.g. fractional knapsack, Kruskal).

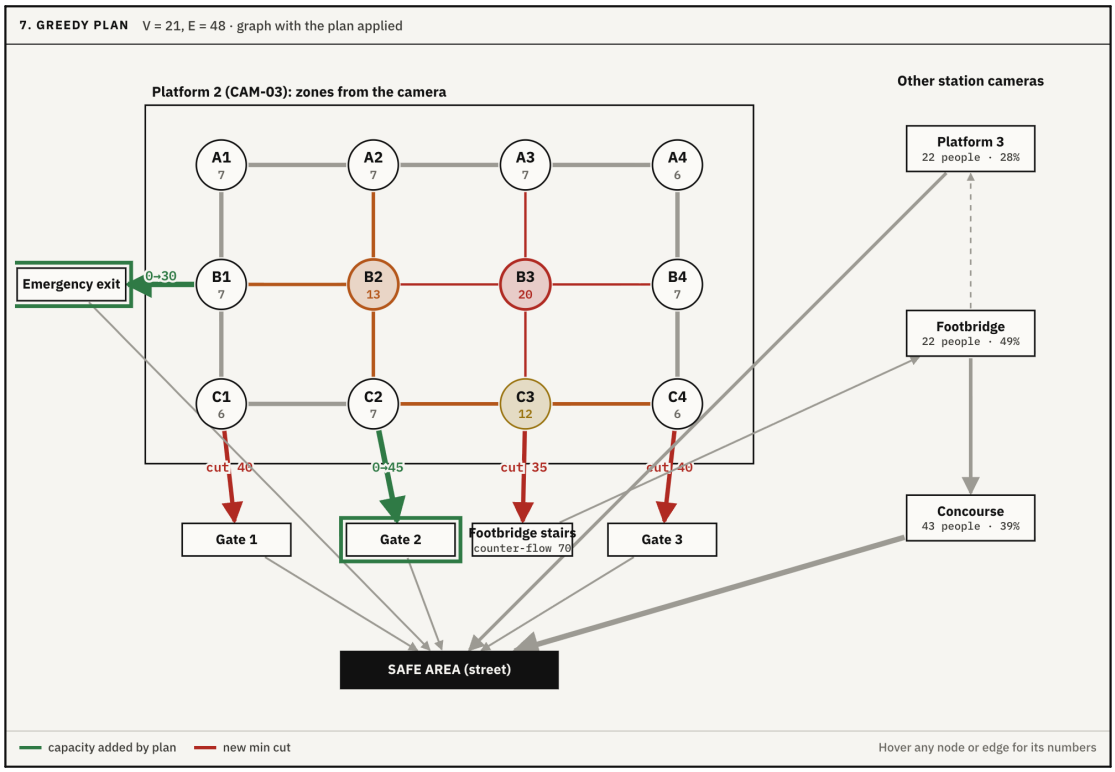
Why we use it in Sentinel. In an emergency, operators need a recommendation in seconds, in a form they can understand: 'do this first, then this'. Sentinel has candidate actions with a cost (staff and disruption points): open Gate 2, pause entry from the footbridge, redirect to Platform 3, open the emergency exit. For each candidate, the greedy step **re-runs max flow** on the graph with that action applied and measures the seconds of clearance time saved. It picks the best **seconds saved ÷ cost**, applies it, and repeats with the new state, until the goal (clear within 35 s) is met or the budget (6 points) runs out.

What it computes here. Clearance starts at 54.8 s. Goal reached after 2 steps at 33.2 s, spending 4 of 6 points.

Step	Option	Cost	Seconds saved	Ratio
1	**Open Gate 2 (chosen)**	2	15.4 s	7.7
	Pause entry from the footbridge	3	9.8 s	3.26
	Redirect to Platform 3	3	0 s	0
	Open platform-end emergency exit	2	11.3 s	5.67
	Pause entry from the footbridge	3	5.3 s	1.77
	Redirect to Platform 3	3	0 s	0
2	**Open platform-end emergency exit (chosen)**	2	6.2 s	3.11

Notice 'Redirect to Platform 3' saves 0 s. It adds capacity after the footbridge, but the footbridge is not in the min cut; the stairs before it are. This is the max-flow min-cut theorem in action, and greedy correctly ignores it.

Greedy is not always optimal. One of our unit tests uses budget 4 with a cheap action (cost 1, +12 capacity) and a big one (cost 4, +40). Greedy takes the cheap one first (better ratio) and can no longer afford the big one; backtracking finds the big one is better.



Graph with the greedy plan applied: green edges gained capacity (Gate 2: 0 → 45, emergency exit: 0 → 30); red edges are the new min cut.

Time	`O(m² · T_flow), T_flow = O(V · E²)`
Space	`O(m)`
Data structure	Array of candidate actions
Observed (this scenario)	1.074 ms per run, averaged over repeated runs
Source file	`src/algorithms/optimization/greedyIntervention.js`

Why not something else? Exhaustive search (backtracking, next section) is exact but exponential. Integer linear programming is exact but heavy for a browser. Greedy is the practical choice for a live decision; we verify it with backtracking.

> *Viva answer: "Greedy gives a fast, explainable plan by repeatedly taking the action with the best benefit-to-cost ratio. It is locally optimal and not guaranteed to be globally optimal, which is why we check it with backtracking."*

4.9 Backtracking: checking the greedy plan

What it is. Backtracking explores a decision tree (here: include or exclude each intervention), and abandons (prunes) a branch as soon as it cannot lead to a valid solution.

> **Why we use it in Sentinel.** We want to be honest about greedy. Backtracking gives the **exact** best plan within the budget, so the app can state whether greedy matched the optimum. With only four actions this is cheap; its exponential growth is exactly why greedy is used for the live decision.

What it computes here. 11 complete plans evaluated, 4 branches pruned (over budget), out of 16 subsets. Best within budget: **Open Gate 2 + Open platform-end emergency exit** (clearance 33.2 s, cost 4). Greedy found the same plan, so greedy matched the optimum here.

Time	`O(2^m · T_flow)`
Space	`O(m) recursion`
Data structure	Recursion (implicit stack)
Observed (this scenario)	1.522 ms per run, averaged over repeated runs
Source file	`src/algorithms/optimization/backtrackingPlanner.js`

Why not something else? Plain brute force evaluates all 2^m subsets with no pruning. Branch and bound with an upper bound on remaining benefit would prune more (Part 8).

> *Viva answer: "Backtracking tries every include/exclude choice and prunes branches that break the budget. It is O(2^m), so we use it to verify greedy, not to decide live."*

4.10 Data structures used

Data structure	Operations and cost	Used by	File
Adjacency list	add edge O(1), neighbours O(degree), memory O(V + E)	Every graph algorithm	`algorithms/graph/Graph.js`
Queue (head index)	enqueue / dequeue O(1)	BFS, Edmonds-Karp, min cut	`dataStructures/Queue.js`
Stack	push / pop O(1)	DFS	`dataStructures/Stack.js`
Binary min-heap	push / pop O(log n), peek O(1)	Dijkstra	`dataStructures/MinHeap.js`
Residual graph	paired arcs i and i XOR 1; update O(1) per arc	Edmonds-Karp, min cut	`flow/maxFlow.js`
Arrays	merge buffers O(n); sorted thresholds	Merge sort, binary search	`sorting/, searching/`

The queue keeps a head index instead of calling Array.shift(), which is O(n) in JavaScript; this keeps BFS truly O(V + E).

PART 5

End-to-end worked example: Emergency Crowd Surge

This is the whole pipeline on one deterministic state, exactly as the professor can trace it in the code: **input → algorithm → output → decision**.

Step	Input	Algorithm	Output	Decision it supports
0	Simulated CCTV at 0:58	Risk engine (CV layer)	105 people, overall CRITICAL, score 94	Start the analysis
1	12 zone densities	Merge sort, binary search	B3 first, CRITICAL	Focus on B3
2	Crowd state + site + side cameras	Graph builder	V = 21, E = 48	-
3	Graph, start B3	BFS	Gate 2 and emergency exit unreachable	Opening an exit is worth considering
4	Graph, start B3	DFS	B3 → B2 → C3	Manage these zones together
5	Graph, costs	Dijkstra ×2	B3 → B4 → C4 → Gate 3	Send B3 towards Gate 3, not the stairs
6	Graph, capacities	Edmonds-Karp	115 people/min, ≈ 55 s	Evacuation is too slow
7	Residual graph	Min cut	Gate 1, stairs (35), Gate 3	Add capacity across the cut
8	4 candidate actions	Greedy (+ max flow per option)	Open Gate 2 + Open platform-end emergency exit	Recommended plan
9	Same candidates	Backtracking	Same plan is optimal	Confidence in the plan
10	Graph with plan	Edmonds-Karp, Dijkstra	190 people/min, ≈ 33 s	Expected improvement
11	Scenario replay	Existing risk model	risk 100 → 25 at 1:18	Independent confirmation

The recommendation the app generates

Zone B3 is critical: 20 people, 111% of crush density.
Current evacuation capacity: 115 people/min. Bottleneck: C1 → Gate 1, C3 → Footbridge stairs, C4 → Gate 3.
Recommended action: Open Gate 2 + Open platform-end emergency exit.
Move people from B3 along B3 → B4 → C4 → Gate 3 → Safe area (street).
Estimated platform clearance: 55 s → 33 s.

Before and after

Measure	Before	After the plan	Change
Evacuation capacity (max flow)	115 /min	190 /min	+65%
Estimated clearance	55 s	33 s	−39%
Bottleneck (min cut)	C1 → Gate 1, C3 → Footbridge stairs, C4 → Gate 3	B1 → Emergency exit, C1 → Gate 1, C2 → Gate 2, C3 → Footbridge stairs, C4 → Gate 3	two more exits share the load
Projected risk at 1:18 (existing risk model)	100/100 CRITICAL	25/100 SAFE	from crush risk to safe

Graph values are recomputed by running the same algorithms on the graph with the plan applied. Projected risk replays the deterministic scenario 20 s ahead with Sentinel's own risk engine, once without and once with the plan. Pressing **Apply recommended plan** adds the same actions to the running simulation: within about 8 seconds the live risk falls from CRITICAL to WATCH.

PART 6

Complexity analysis, comparison and benchmark

Algorithm	Sentinel use	Time	Space	Observed
Merge sort	Rank zones from most to least dangerous	`O(n log n)`	`O(n)`	`1.6 µs`
Binary search	Find each zone's alert band in the sorted thresholds	`O(log n)`	`O(1)`	`0.7 µs`
Graph construction	Turn the crowd state into a weighted graph	`O(V + E)`	`O(V + E)`	`0.049 ms`
Breadth-first search	Which exits can the critical zone still reach?	`O(V + E)`	`O(V)`	`0.011 ms`
Depth-first search	Which connected zones are affected by the dangerous zone?	`O(V + E)`	`O(V)`	`1.5 µs`
Dijkstra	Safest evacuation route from the critical zone	`O((V + E) log V)`	`O(V + E)`	`0.011 ms`
Max flow (Edmonds-Karp)	Evacuation capacity in people per minute	`O(V · E²)`	`O(V + E)`	`0.083 ms`
Min cut	Bottleneck that limits evacuation	`O(V + E) after max flow`	`O(V)`	`0.013 ms`
Greedy intervention selection	Fast plan: best benefit / cost first	`O(m² · T_flow), T_flow = O(V · E²)`	`O(m)`	`1.074 ms`
Backtracking planner	Exact best plan within the budget (checks greedy)	`O(2^m · T_flow)`	`O(m) recursion`	`1.522 ms`

V = 21, E = 48 edges in the scenario graph; m = 4 candidate interventions; T_flow = one Edmonds-Karp run. Observed times are averages over repeated runs on one machine (Node.js), measured with performance.now().

Deriving the bounds

- **BFS / DFS: O(V + E).** Every node enters the queue/stack at most once (visited set) and every adjacency list is scanned once.

- **Dijkstra: O((V + E) log V).** Each node is settled once (V pops); each edge relaxation may push one heap entry (E pushes); each heap operation is O(log E) = O(log V).
- **Edmonds-Karp: O(V·E²).** Each BFS is O(E). With shortest augmenting paths, the BFS distance to every node never decreases, and each edge can be the bottleneck (critical) at most O(V) times, so there are O(V·E) augmentations.
- **Min cut: O(V + E)** once the max flow is known: one BFS on the residual graph and one pass over the edges.
- **Greedy: O(m²·T_flow)** worst case: at most m steps, each evaluating up to m options with one max-flow run each. Here that is at most 4 + 3 + 2 + 1 = 10 max-flow runs.
- **Backtracking: O(2^m·T_flow)** worst case: 16 subsets for m = 4; pruning removed 4 branches here. At m = 30 it would be over a billion.
- **Whole pipeline:** dominated by greedy and backtracking. For this station that is about 1–2 ms in total.

Algorithm comparisons

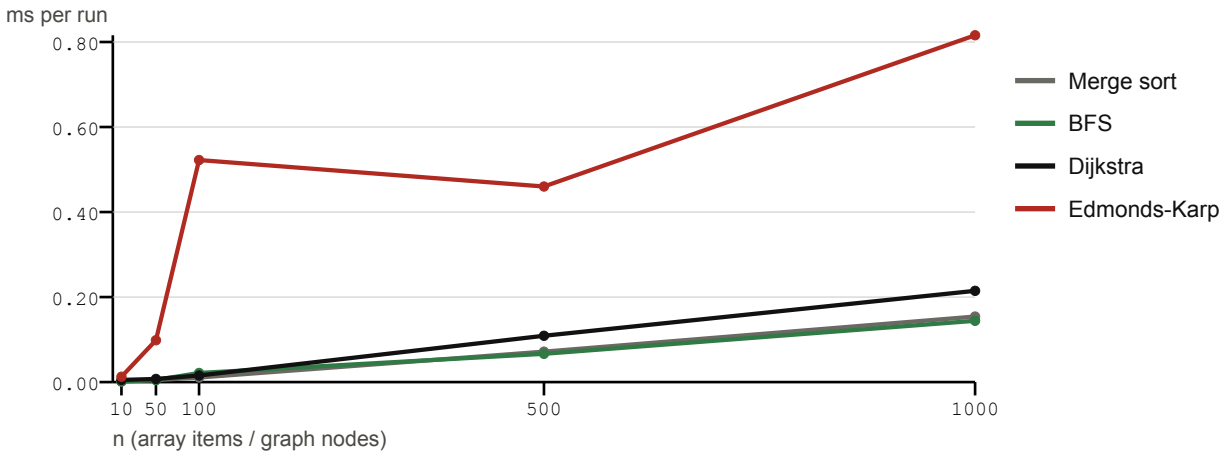
Comparison	What it shows
BFS vs Dijkstra	BFS finds the fewest hops; Dijkstra the cheapest path when edges have different non-negative weights.
Dijkstra on metres vs on cost	Same algorithm, different weights: the safest route is 12 m longer but avoids the crush.
Ford-Fulkerson vs Edmonds-Karp	Arbitrary paths: O(E·f), depends on capacities. Shortest (BFS) paths: O(V·E²), depends only on graph size.
Max flow vs min cut	Equal by the theorem; the app and the tests check that the cut capacity equals the flow.
Greedy vs backtracking	Polynomial and locally optimal vs exponential and exact. Here they agree; a unit test shows a case where they do not.
Merge sort vs quicksort	Both O(n log n) on average; merge sort is also O(n log n) in the worst case and stable.

Theoretical complexity vs observed runtime

Big-O is a worst-case bound on growth; a stopwatch is a single observation on one machine. The Algorithm Analysis tab shows both side by side and states clearly that observed runtimes **cannot prove** a bound. Benchmark mode runs merge sort, BFS, Dijkstra and Edmonds-Karp on seeded synthetic inputs (random arrays, grid graphs) of size 10 to 1000:

n	E	Merge sort (ms)	BFS (ms)	Dijkstra (ms)	Edmonds-Karp (ms)
10	26	0.0011	0.0018	0.0051	0.0126
50	170	0.0047	0.0050	0.0074	0.0985
100	360	0.0107	0.0214	0.0154	0.5223
500	1910	0.0716	0.0665	0.1089	0.4601
1000	3872	0.1541	0.1442	0.2147	0.8158

Measured when this report was generated (Node.js). Merge sort, BFS and Dijkstra grow roughly in line with n log n, V + E and (V + E) log V. Edmonds-Karp grows far more slowly than its O(V·E²) worst case on these grids, because the number of augmenting paths stays small; worst-case bounds are often pessimistic in practice. Small sizes are noisy because of timer resolution and JIT warm-up.



Observed runtime per run against input size (numeric axis, so the curve shapes reflect real growth). Same data as the table.

PART 7

Testing and how to verify the results

The UI is not the only place the algorithms are checked. **37 automated tests** run with Node's built-in test runner (npm test), with no extra libraries:

Area	What is tested
Data structures	Queue FIFO with 200 items and empty dequeue; stack LIFO; min-heap pops in priority order
Merge sort	Normal, empty, single item, descending comparator, stability, 60 random arrays against a reference sort, trace has n − 1 merges
Binary search	Every present item, missing item, empty and single-element arrays; band lookup at every boundary; probe count ≤ log n + 1
BFS / DFS	Hop order, single node, missing start, disconnected graph, closed (zero-capacity) edge, cycles, DFS filter
Dijkstra	Cheaper-not-fewer-hops route, distance vs cost weights, unreachable target, closed edge, negative weight rejected, 30 random graphs checked against Floyd-Warshall
Max flow / min cut	CLRS textbook network (23), multiple exits with a bottleneck corridor, flow conservation, zero-capacity edge, disconnected graph, reverse edges undoing a bad path, infinite-capacity guard, cut capacity = flow
Greedy / backtracking	Ratio order, budget, goal, empty and useless candidates, a case where greedy is not optimal, subset and pruning counts
End to end	Scenario determinism; graph built from the crowd state; full analysis invariants; applied actions not re-offered; empty crowd; no exits configured; every action logged exactly once

Integrity rules we followed

- No routing, graph or sorting library; Array.prototype.sort is not used for the ranking.
- No hard-coded algorithm outputs: every number in the app and in this report is computed from the graph (this report reads a JSON file exported from the engine).
- No random numbers in the demo scenario; random inputs only in tests and the benchmark, always with a fixed seed.
- Physical constants (exit widths, walking distances, which gate is locked) are a clearly labelled dataset in engine/stationModel.js.
- Observed runtimes are labelled as observations, never as proof of complexity.

How to trace a result in the code

Result	Follow this path
Critical zone	`decisionEngine.js runDaaAnalysis → riskEngine.js rankZones → mergeSort.js, binarySearch.js`
Safest route	`decisionEngine.js → crowdGraphBuilder.js buildCrowdGraph (edge costs) → dijkstra.js (MinHeap.js)`
Evacuation capacity	`decisionEngine.js evacuationCapacity → crowdGraphBuilder.js withCrowdSource → maxFlow.js (Queue.js)`
Bottleneck	`decisionEngine.js → minCut.js on the residual graph from maxFlow.js`
Plan	`decisionEngine.js → greedyIntervention.js with objective = clearance from maxFlow; check in backtrackingPlanner.js`

PART 8

What we will add next

These additions are **planned, not implemented**. Each one extends a DAA idea already in Sentinel, and each answers a limitation we state openly.

8.1 Dynamic programming: allocating people across exits

Problem. Max flow gives the total capacity, but not 'how many people from B3 should go to each gate' so that the last person is out as early as possible. With k exits, exit j having capacity c_j (people/min) and walking time w_i, assigning x_i people to it finishes at $w_i + x_i / c_i$. We want to minimise the latest finish.

```
dp[j][p] = minimum possible latest finish time using exits 1..j for p people
dp[0][0] = 0, dp[0][p] = infinity for p > 0
dp[j][p] = min over x = 0..p of max( dp[j-1][p-x], w_j + x / c_j )
answer = dp[k][n]; time O(k * n^2), space O(k * n)
```

Why DP: the problem has optimal substructure (the best split of the remaining people over the remaining exits does not depend on how earlier exits were filled) and overlapping subproblems. **Why not now:** we chose not to ship it until it is fully tested, rather than show a fake result.

8.2 Other algorithmic additions

Addition	DAA concept	Why it helps Sentinel	Complexity
Min-cost max-flow	Successive shortest paths with potentials	Moves the maximum number of people while also minimising total route cost: combines Dijkstra and max flow in one answer	$O(F \cdot E \log V)$
Flow over time	Time-expanded network	Real clearance times that include walking time and queues, not just people ÷ flow	max flow on $V \cdot T$ nodes
Dinic's algorithm / push-relabel	Blocking flows, level graphs	Faster max flow for stadium-size graphs with thousands of nodes	$O(V^2 \cdot E)$ / $O(V^3)$
A* and D* Lite	Heuristic and incremental shortest paths	Faster routing on large maps; update routes every tick instead of recomputing from scratch	≤ Dijkstra; incremental
Union-Find	Disjoint sets with path compression	Keep danger clusters up to date as zones change level, instead of re-running DFS	≈ $O(\alpha(n))$ per update
Hungarian algorithm	Assignment problem / bipartite matching	Assign stewards to critical zones and gates to minimise total response time	$O(n^3)$
Branch and bound	Bounding + pruning	Exact intervention plans for larger action sets, pruning with an upper bound on remaining benefit	exponential worst case, far fewer nodes
Sliding window with deque	Monotonic queue	O(1) per tick surge and maximum detection over long histories	O(1) amortised

8.3 System additions

- **Crowd-density model** (e.g. CSRNet-style density maps) for very dense crowds, where a person detector undercounts.
- **Multi-camera fusion:** real cameras for every station area instead of simulated side cameras, feeding the same graph.
- **Calibrated capacities:** measure real people-per-minute at gates and stairs instead of illustrative constants.
- **Alerts to staff phones** and a small backend for logs and multi-operator use (SMS is simulated today).
- **Site editor for the graph:** draw corridors and capacities for any venue, not only the demo station.

Known limitations today

- Clearance = people ÷ max flow is a steady-flow estimate that ignores walking time (fixed by flow over time, 8.2).
- Capacities and action costs are illustrative constants for one demo station.
- Every occupied zone is fed by an unlimited virtual source, so max flow measures exit capacity, not per-zone demand.
- Projected risk is only available in Simulation mode; a live camera cannot be replayed into the future.

APPENDIX A

30-minute demo script

Time	Show	Say
0–3	Landing page	The crowd-crush problem; CV tells us what is happening, DAA tells us what to do.
3–7	Monitor, Station tabs (simulation; optionally Live with a demo video)	Zones, counts, movement, surge, turbulence, counter-flow; explainable alerts.
7–10	DAA Engine: live graph → Load Emergency Crowd Surge → Run DAA Analysis	How the crowd becomes a graph: nodes, edges, capacity, cost. Steps 1–2.
10–14	Steps 3–5 (use Replay)	BFS: Gate 2 unreachable. DFS: danger cluster. Dijkstra: safest ≠ shortest, and why.
14–20	Step 6	Max flow 115/min, three augmenting paths, residual graph, min cut, counter-flow on the stairs.
20–24	Step 7	Greedy ratio table; redirect saves 0 s (not on the cut); backtracking confirms optimum; greedy not always optimal.
24–27	Step 8 → Apply recommended plan; Monitor	Before/after: 115 → 190/min, 55 s → 33 s; live risk drops to WATCH.
27–30	Algorithm Analysis → Run benchmark	Theory vs observed; data structures; trade-offs; what we will add.

Keyboard: 1–7 switch tabs, ← → move between DAA steps, Space pause/play, R restart, F fullscreen.

APPENDIX B

Likely viva questions

Q. Why is Dijkstra correct here?

All edge costs are distance, congestion and risk multiplied by positive weights, so they are never negative. With non-negative weights, the node with the smallest tentative distance cannot be improved later, so settling it is safe.

Q. Why Edmonds-Karp and not plain Ford-Fulkerson?

Ford-Fulkerson's running time depends on the flow value; Edmonds-Karp chooses shortest augmenting paths with BFS, which bounds augmentations to $O(V \cdot E)$ and the total to $O(V \cdot E^2)$, independent of capacities.

Q. What is the residual graph and why do we need reverse edges?

It records how much more flow each edge can take (forward) and how much can be undone (reverse). Without reverse edges an early bad path could block the optimum.

Q. How do you get the min cut from the max flow?

After the last BFS fails, the nodes reachable from the source in the residual graph form side S. Original edges from S to the rest are saturated, and their capacities sum to the max flow.

Q. Why does 'Redirect to Platform 3' give zero benefit?

It adds capacity after the footbridge, but the min cut is at the stairs before it. Only increasing capacity across the min cut can raise the maximum flow.

Q. Is the greedy plan optimal?

Not in general. Here backtracking checked all 16 subsets and found the same plan. A unit test shows a budget where greedy is worse.

Q. Why merge sort instead of the built-in sort?

To implement and control the algorithm: guaranteed O(n log n) worst case and stability. The built-in sort is engine-dependent and would hide the algorithm.

Q. What is the overall complexity of one analysis?

Dominated by greedy (O(m²) max-flow runs) and backtracking (O(2^m) runs), each Edmonds-Karp O(V·E²). With V = 21, E = 48 and m = 4, about 1–2 ms.

Q. Do your runtime measurements prove the complexity?

No. They are observations on one machine; Big-O is a mathematical bound. The benchmark only illustrates trends.

Q. Where do the numbers come from?

From the crowd state (camera or deterministic simulation) and a labelled station dataset; every result is computed by the algorithms and can be traced in decisionEngine.js.

Q. What if the camera fails during the demo?

The DAA engine is independent of computer vision. Simulation mode and the emergency scenario run offline, with no model, webcam or internet.

Q. What would you add with more time?

Dynamic programming for exit allocation, min-cost max-flow, flow over time for real clearance times, and Dinic's algorithm for large venues.

APPENDIX C

How to run

```
npm install
npm run dev (open http://localhost:5173)
npm test (37 tests)
npm run build
```

In the app: **DAA Engine** tab → **Load Emergency Crowd Surge** → **Run DAA Analysis**. For Live detection, use the clips in the sentinel-demo-videos folder (Live detection → Upload MP4); the AI model (≈ 67 MB) downloads once and is then cached.

This report was generated from the code: docs/report/export-data.mjs exports the engine's results, capture.mjs takes the screenshots, and make_report.py builds the PDF.